QUESTIONS

1. Define Pytorch and how does it different from other deep learning frameworks like tensorflow ?
2. What is the use of .dtype , .shape and .device in pytorch ?
3. What is the difference between @ and * operations between tensors ?
4. What is the use of torch.bmm() ?
5. What is the use of .reshape() ?
6. What is the use of torch.where() ?
7. What is the use of torch.argmax() and torch.topk() ?
8. What is the use of torch.gather() ?
9. What is AutoGrad ? How we can track the operations performed on any tensor ?
10. What is computation graph ? How to Check the computation graph ?
11. What is the use of .backward() ?
12. What is one problem of AutoGrad ? And How to resolve that problem ?
13. Difference Between .cpu(), .cuda() and .to(device) ?
14. Difference Between .item() and .tolist() ?
15. What Happens If You Don’t Use torch.no_grad() During Inference ?
16. Difference Between .backward() and torch.autograd.grad() ?

THEORY

1. PyTorch is a dynamic, Pythonic deep learning framework developed by Facebook AI, known for flexibility and research usability.

   TensorFlow is also a deep learning framework developed by Google, originally used static graphs for optimized deployment but now supports eager execution.

   PyTorch is preferred in research, while TensorFlow has strong production deployment tools.

2. .shape : This is a tuple that describes the dimensions of the tensor. torch.Size([2, 3]) tells us it's a 2D tensor with 2 rows and 3 columns.

.device : This tells you where the tensor's data is physically stored. By default, it's on the cpu. If you have a compatible GPU, you can move it there (.to("cuda")) for massive speedups.

.dtype : This describes the data type of the numbers inside the tensor. Notice it defaulted to torch.float32

3. "*" performs element-wise multiplication between tensors of the same shape (or broadcastable shape).

```
Key Points:

Shape must be same or broadcastable.

it doesn't perform matrix multiplication.

it is used in attention masking, gating, scaling.
```

@ performs matrix multiplication (dot product for 1D, matrix multiply for 2D, batch matrix multiply for 3D). And it is equivalent to torch.matmul().

Dimensional Behaviour -

Tensor Dimension	`*`	`@`
1D	Element-wise	Dot product
2D	Element-wise	Matrix multiplication
3D	Element-wise	Batch matrix multiplication

4. torch.bmm() performs batch matrix multiplication but only works on 3D tensors and does not support broadcasting.

```
example -
a = torch.randn(5, 2, 3)
b = torch.randn(5, 3, 4)
output = torch.bmm(a, b)
print(output.shape)   # (5, 2, 4)
```

5. torch.reshape() returns a tensor with the same data but a new shape, without changing the values.

   and its syntax is -

   torch.reshape(input, new_shape)

6. torch.where() is a PyTorch function used to select elements from tensors based on a condition, operating element-wise and fully compatible with autograd.

syntax is torch.where(condition, value_if_true, value_if_false)

7. After a model makes a prediction, we often need to find the item with the highest score.

```
torch.argmax(): Finds the index of the single maximum value.

torch.topk(): Finds the k largest values and their indices.
```

8. In PyTorch, torch.gather() is used to select values from a tensor using index positions along a specific dimension.wise conditional selection between two tensors based on a boolean condition.

```
its syntax is - torch.gather(input, dim, index)

It gathers values from input along dimension dim using index.

Example -
import torch

input = torch.tensor([[10, 20, 30],
                      [40, 50, 60]])

index = torch.tensor([[0, 2],
                      [1, 0]])

output = torch.gather(input, dim=1, index=index)
print(output)

and output will be:

tensor([[10, 30],
 [50, 40]])
```

9. Autograd is PyTorch’s automatic differentiation engine that records the operations performed on tensors during the forward pass and builds a dynamic computation graph.

   To track the operations performed on any tensor we must use "requires_grad = True" as parameter while defining that tensor.

   By default, PyTorch assumes a tensor is just static data. To tell it that a tensor is a learnable parameter (like a model's weight or bias), you must set its requires_grad attribute to True.

10. Computation Graph is a directed acyclic graph that represents the sequence of operations performed on tensors during the forward pass. And it is used to compute gradients during backpropagation.

to check the computation graph or the operations performed on a tensor we can use .grad_fn.

11. .backward() is used to compute gradients of a tensor with respect to all tensors that have "requires_grad = True" and stores them in the ".grad" attribute of leaf tensors.

12. One major issue with Autograd is that gradients accumulate by default. Each call to " .backward() " adds gradients to the existing " .grad values " instead of replacing them. To fix this, we must call " optimizer.zero_grad() " before the backward pass to clear old gradients.

```
because accumulation of gradients will cause:

> Wrong updates

> Unstable training

> Exploding gradients

> Silent failures
```

hence, to prevent this all we follow this in our training loop:

optimizer.zero_grad() - clear old gradients

loss.backward() - calculate gradients

optimizer.step() - update weights

```
and we use optimizer.zero_grad() before the loss.backward()
because we want to clear gradients from the previous iteration
before computing new ones.
```

13. .cpu() moves a tensor to CPU, .cuda() moves it to GPU, and .to(device) is a general method that moves tensors to a specified device. .to(device) is preferred because it makes code portable.

14. .item() -

    Converts a single-element tensor into Python scalar

    Works only for one-element tensors

```
example -

x = torch.tensor(5)
x.item()  # 5
```

.tolist() -

Converts tensor to Python list

Works for multi-element tensors

```
example -

x = torch.tensor([1, 2, 3])
x.tolist()  # [1, 2, 3]
```

15. If torch.no_grad() is not used during inference, PyTorch will track gradients and build a computation graph, leading to unnecessary memory usage and slower execution.

```
hence, correct inference pattern -

model.eval()

with torch.no_grad():

outputs = model(inputs)
```

16. " .backward() " computes gradients and stores them in .grad, while " torch.autograd.grad() " computes and returns gradients explicitly without modifyin .grad.

CODING QUESTIONS

1. How to install and use pytorch on various platforms ?
2. How to check whether our installation recognizes our built-in NVIDIA GPU or not ?
3. What are the various methods to create tensors ?
4. How to check the datatype, shape and device of any tensor ?
5. Perform basic mathematical operations on tensor like:

```
i) element-wise multiplication and matrix multiplication

ii) calculate sum, mean, maximum, minimum values using "dim"
parameter

iii) change the shape of tensor without changing the values of
tensor

iv) transpose the tensor
```

6. Perform Indexing and Selcetion operations like:

```
    i) Standard Indexing: The Basics

    ii) Boolean Masking: Selection by Condition

    iii) Conditional Creation using torch.where()

    iv) Primitives for Finding the "Best" Items using torch.argmax()
    and torch.topk()

    v) Primitive for Dynamic Lookups using torch.gather()
```

7. AUTOGRAD

```
# HOW TO INSTALL PYTORCH

!pip install torch
import torch
torch.__version__
```

```
Requirement already satisfied: torch in
/usr/local/lib/python3.12/dist-packages (2.9.0+cpu)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

{"type":"string"}

```
# How to check whether our installation recognizes our built-in NVIDIA
GPU
# if it returns True , then it means it recognizes built-in NVIDIA GPU
# if it returns False , then no.

torch.cuda.is_available()
```

```
False
```

Various methods to create tensors

1. Basic
2. From python lists
3. Creation from a Desired Shape
4. Creation by Mimicking Another Tensor's Properties

```
# BASIC

# create a 0D tensor (scalar) from a Python integer
tensor0d = torch.tensor(1)

# create a 1D tensor (vector) from a Python list
tensor1d = torch.tensor([1, 2, 3])

# create a 2D tensor from a nested Python list
tensor2d = torch.tensor([[1, 2], [3, 4]])

# create a 3D tensor from a nested Python list
tensor3d = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(tensor0d)
print(tensor1d)
print(tensor2d)
print(tensor3d)
```

```
tensor(1)
tensor([1, 2, 3])
tensor([[1, 2],
        [3, 4]])
tensor([[[1, 2],
         [3, 4]],

        [[5, 6],
         [7, 8]]])
```

```
# This is the most straightforward. You have a Python list of numbers,
and you want to convert it into a PyTorch tensor using torch.tensor().

import torch

# Input: A standard Python list
data = [[1, 2, 3], [4, 5, 6]]
my_tensor = torch.tensor(data)

print(my_tensor)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

```python
# Creation from desired shape

# Input: A shape tuple (2 rows, 3 columns)
shape = (2, 3)

ones = torch.ones(shape)
zeros = torch.zeros(shape)
random = torch.randn(shape)

print(f"Ones Tensor:\n {ones}\n")
print(f"Zeros Tensor:\n {zeros}\n")
print(f"Random Tensor:\n {random}")
```

```
Ones Tensor:
 tensor([[1., 1., 1.],
        [1., 1., 1.]])

Zeros Tensor:
 tensor([[0., 0., 0.],
        [0., 0., 0.]])

Random Tensor:
 tensor([[-1.0168,  0.5139,  0.3082],
        [ 0.6723,  1.3068, -0.6924]])
```

```python
# Often, you have an existing tensor and you need to create a new one
that has the exact same shape, data type, and is on the same device.
# Instead of manually copying these properties, PyTorch provides handy
_like functions.


template = torch.tensor([[1, 2], [3, 4]])

# Create new tensors with the same properties as the template
ones_like = torch.ones_like(template)
rand_like = torch.randn_like(template, dtype=torch.float) # dtype can
be overridden

print(f"Template Tensor:\n {template}\n")
print(f"Ones_like Tensor:\n {ones_like}\n")
print(f"Randn_like Tensor:\n {rand_like}")
```

```
Template Tensor:
 tensor([[1, 2],
        [3, 4]])

Ones_like Tensor:
 tensor([[1, 1],
        [1, 1]])

Randn_like Tensor:
```

```
 tensor([[-1.0020,  0.6236],
        [ 0.0717, -0.4501]])
```

Shape, Type, and Device of tensor

```
# Every tensor has attributes that describe its metadata
with .shape, .dtype, and .device.

# Let's create a tensor to inspect
tensor = torch.randn(2, 3)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")
```

```
Shape of tensor: torch.Size([2, 3])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

Basic Mathematical & Reduction Operations

```
# Mathematical Operations: * vs. @


a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[10, 20], [30, 40]])

# Calculation: [[1*10, 2*20], [3*30, 4*40]]
element_wise_product = a * b

print(f"Tensor a:\n {a}\n")
print(f"Tensor b:\n {b}\n")
print(f"Element-wise Product (a * b):\n {element_wise_product}")

m1 = torch.tensor([[1, 2, 3], [4, 5, 6]])   # Shape: (2, 3)
m2 = torch.tensor([[7, 8], [9, 10], [11, 12]]) # Shape: (3, 2)

# Calculation for the first element: (1*7) + (2*9) + (3*11) = 58
matrix_product = m1 @ m2 # Resulting shape: (2, 2)

print(f"Matrix 1 (shape {m1.shape}):\n {m1}\n")
print(f"Matrix 2 (shape {m2.shape}):\n {m2}\n")
print(f"Matrix Product (m1 @ m2):\n {matrix_product}")
```

```
Tensor a:
 tensor([[1, 2],
        [3, 4]])
```

```
Tensor b:
 tensor([[10, 20],
        [30, 40]])

Element-wise Product (a * b):
 tensor([[ 10,  40],
        [ 90, 160]])
Matrix 1 (shape torch.Size([2, 3])):
 tensor([[1, 2, 3],
        [4, 5, 6]])

Matrix 2 (shape torch.Size([3, 2])):
 tensor([[ 7,  8],
        [ 9, 10],
        [11, 12]])

Matrix Product (m1 @ m2):
 tensor([[ 58,  64],
        [139, 154]])
```

```python
# Batch Multiplication
import torch

A = torch.randn(4, 3, 5)   # batch=4
B = torch.randn(4, 5, 2)

C = torch.bmm(A, B)
print(C.shape)  # torch.Size([4, 3, 2])
```

```
torch.Size([4, 3, 2])
```

```python
# Reduction Operations include sum(), mean(), max(), and min()

scores = torch.tensor([[10., 20., 30.], [5., 10., 15.]])

# By default, reductions apply to the entire tensor
total_sum = scores.sum()
average_score = scores.mean()

print(f"Scores Tensor:\n {scores}\n")
# Calculation: 10 + 20 + 30 + 5 + 10 + 15 = 90
print(f"Total Sum: {total_sum}")
# Calculation: 90 / 6 = 15
print(f"Overall Mean: {average_score}")
```

```
Scores Tensor:
 tensor([[10., 20., 30.],
        [ 5., 10., 15.]])
```

```
Total Sum: 90.0
Overall Mean: 15.0
```

```python
# dim Argument

scores = torch.tensor([[10., 20., 30.], [5., 10., 15.]])

# To get the sum FOR EACH ASSIGNMENT, we collapse the student
dimension (dim=0)
# Calculation: [10+5, 20+10, 30+15]
sum_per_assignment = scores.sum(dim=0)

# To get the sum FOR EACH STUDENT, we collapse the assignment
dimension (dim=1)
# Calculation: [10+20+30, 5+10+15]
sum_per_student = scores.sum(dim=1)

print(f"Original Scores:\n {scores}\n")
print(f"Sum per assignment (dim=0): {sum_per_assignment}")
print(f"Sum per student (dim=1):    {sum_per_student}")
```

```
Original Scores:
 tensor([[10., 20., 30.],
        [ 5., 10., 15.]])

Sum per assignment (dim=0): tensor([15., 30., 45.])
Sum per student (dim=1):    tensor([60., 30.])
```

```python
# tensor.reshape()

# Original shape: (2, 3)
# New shape: (3, 2)
# Total elements: 6 → must match

x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

y = x.reshape(3, 2)
print(y)

# PyTorch can infer one dimension automatically.
x1 = torch.randn(12)

y1 = x1.reshape(3, -1)
print(y1.shape)  # torch.Size([3, 4])
```

```
tensor([[1, 2],
        [3, 4],
        [5, 6]])
torch.Size([3, 4])
```

```python
# tensor.view()

x = torch.arange(6)
print(x)
# tensor([0, 1, 2, 3, 4, 5])

y = x.view(2, 3)
print(y)

# tensor.T

x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

y = x.T
print(y)
```

```
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

Indexing & Selection Primitives

```python
# Standard Indexing: The Basics

# A simple, easy-to-read tensor
x = torch.tensor([[0, 1, 2, 3],
                  [4, 5, 6, 7],
                  [8, 9, 10, 11]])

# Get the second row (at index 1)
row_1 = x[1]
print(f"Row 1: {row_1}\n")

# Get the third column (at index 2)
col_2 = x[:, 2]
print(f"Column 2: {col_2}\n")

# Get a specific element (row 1, column 3)
element_1_3 = x[1, 3]
print(f"Element at (1, 3): {element_1_3}")
```

```
Row 1: tensor([4, 5, 6, 7])

Column 2: tensor([ 2,  6, 10])

Element at (1, 3): 7
```

```python
# Boolean Masking: Selection by Condition

x = torch.tensor([[0, 1, 2, 3], [4, 5, 6, 7]])

# Step 1: Create the boolean mask
mask = x > 3
print(f"The Boolean Mask (x > 3):\n {mask}\n")

# Step 2: Apply the mask to the tensor
selected_elements = x[mask]
print(f"Selected elements: {selected_elements}")
```

```
The Boolean Mask (x > 3):
 tensor([[False, False, False, False],
        [ True,  True,  True,  True]])

Selected elements: tensor([4, 5, 6, 7])
```

```python
# Conditional Creation: torch.where()

x = torch.tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Create a new tensor: if a value in x is > 4, use -1, otherwise use
the original value
y = torch.where(x > 4, -1, x)

print(f"Original Tensor: {x}")
print(f"Result of where: {y}")
```

```
Original Tensor: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
Result of where: tensor([ 0,  1,  2,  3,  4, -1, -1, -1, -1, -1])
```

```python
# Primitives for Finding the "Best" Items

scores = torch.tensor([
    [10, 0, 5, 20, 1],  # Scores for item 0
    [1, 30, 2, 5, 0]   # Scores for item 1
])

# Find the index of the best score for each item
best_indices = torch.argmax(scores, dim=1)
# For row 0, max is 20 at index 3. For row 1, max is 30 at index 1.
print(f"Argmax indices: {best_indices}\n")

# Find the top 3 scores and their indices for each item
top_values, top_indices = torch.topk(scores, k=3, dim=1)
print(f"Top 3 values:\n {top_values}\n")
print(f"Top 3 indices:\n {top_indices}")
```

```
Argmax indices: tensor([3, 1])
```

```
Top 3 values:
 tensor([[20, 10,  5],
        [30,  5,  2]])

Top 3 indices:
 tensor([[3, 0, 2],
        [1, 3, 2]])
```

```
# Primitive for Dynamic Lookups: torch.gather()

data = torch.tensor([
    [10, 11, 12, 13],  # row 0
    [20, 21, 22, 23],  # row 1
    [30, 31, 32, 33]   # row 2
])

# Our "personalized list" of column indices to select from each row
indices_to_select = torch.tensor([[2], [0], [3]])

# Gather from `data` along dim=1 (the column dimension)
selected_values = torch.gather(data, dim=1, index=indices_to_select)

print(f"Selected Values:\n {selected_values}")
```

```
Selected Values:
 tensor([[12],
        [20],
        [33]])
```

AUTOGRAD

```
# A standard data tensor (we don't need to calculate gradients for our
input data)
x_data = torch.tensor([[1., 2.], [3., 4.]])
print(f"Data tensor requires_grad: {x_data.requires_grad}\n")

# A parameter tensor (we need to learn this, so we need gradients)
w = torch.tensor([[1.0], [2.0]], requires_grad=True)
print(f"Parameter tensor requires_grad: {w.requires_grad}")
```

```
Data tensor requires_grad: False

Parameter tensor requires_grad: True
```

Once you set requires_grad=True, PyTorch starts building a computation graph behind the scenes. Think of it as a history of all the operations. Every time you perform an operation on a tensor that requires gradients, PyTorch adds a new node to this graph.

```
# Three parameter tensors that we want to learn
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(4.0, requires_grad=True)

# First operation: y = a + b
y = a + b

# Second operation: z = x * y
z = x * y

print(f"Result of a + b: {y}")
print(f"Result of x * y: {z}")
```

```
Result of a + b: 5.0
Result of x * y: 20.0
```

How can we prove this graph exists? Every tensor that is the result of an operation on a requires_grad tensor will have a special attribute called .grad_fn.

```
# z was created by multiplication
print(f"grad_fn for z: {z.grad_fn}")

# y was created by addition
print(f"grad_fn for y: {y.grad_fn}")

# a was created by the user, not by an operation, so it has no grad_fn
print(f"grad_fn for a: {a.grad_fn}")
```

```
grad_fn for z: <MulBackward0 object at 0x7b372a076e60>
grad_fn for y: <AddBackward0 object at 0x7b37368cc250>
grad_fn for a: None
```

BASIC AUTOGRAD USING SCALAR

```
x = torch.tensor(4.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

# forward pass
z = x**2 + y**3
print(f"z = {x}^2 + {y}^3 = {z}")

# backward pass: compute gradients

z.backward()
dz_dx = x.grad # partial derivative wrt x
dz_dy = y.grad # partial derivative wrt y

print(f"dz/dx = {dz_dx} and dz/dy = {dz_dy}")
```

```
z = 4.0^2 + 5.0^3 = 141.0
dz/dx = 8.0 and dz/dy = 75.0
```

```python
# chain rule
x = torch.tensor(4.0, requires_grad=True)
y = x**2
z = y+1
w = z**2
print(f"x = {x} \ny = x^2 = {y} \nz = y+1 = {z} \nw = z^2 = {w}")

w.backward()

print(f"dw/dx = {x.grad}")
```

```
x = 4.0
y = x^2 = 16.0
z = y+1 = 17.0
w = z^2 = 289.0
dw/dx = 272.0
```

One Problem Of Autograd Is the Accumulation Of Gradients.

see with example

```python
x = torch.tensor(2.0, requires_grad=True)

y = x * x
y.backward()

print(x.grad)  # 4

y = x * x
y.backward()

print(x.grad)
```

```
tensor(4.)
tensor(8.)
```

Q. Explain nn.module ?

Q. Explain nn.linear and nn.embedding ?

Q. Explain the forward () method ?

Q. Write the code of FFN in Pytorch ?

Q. Explain Learning Rate and Gradient Clipping method ?

PyTorch nn.Module

The base class for all neural network modules in PyTorch.

1. Definition

`nn.Module` is the base class that all neural network models inherit from. It provides:

- Parameter registration
- Layer management
- GPU/CPU transfer
- Training / evaluation modes
- Saving and loading models

2. Syntax

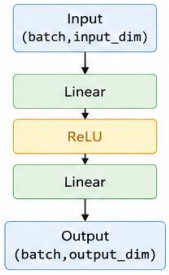
```
class MyModel(nn.Module):
    def __init__(self):
        super(MyModel, self).__init__()
        # define layers

    def forward(self, x):
        # define computation
        return x
```

3. Key Features

- Automatically tracks parameters (`nn.Parameter`)
- Contains submodules (layers)
- Easy to move model to GPU/CPU
- `model.train()` / `model.eval()`
- Save / load entire model

4. Example

```
import torch
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(SimpleNet, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```



5. Useful Methods

`model.parameters()`	Returns all parameters
`model.named_parameters()`	Returns names with parameters
`model.children()`	Returns immediate submodules
`model.train()`	Sets model to training mode
`model.eval()`	Sets model to evaluation mode
`model.to(device)`	Moves model to device (GPU/CPU)
`model.state_dict()`	Returns model parameters (dict)
`model.load_state_dict()`	Loads parameters from state dict

6. Saving & Loading

```
# Save
torch.save(model.state_dict(), "model.pth")

# Load
model = SimpleNet(10, 64, 2)
model.load_state_dict(torch.load("model.pth"))
model.eval()
```

7. Parameter Registration

Any layer (`nn.Module`) or parameter (`nn.Parameter`) assigned as an attribute is automatically registered.

```
self.weight = nn.Parameter(torch.randn(10, 10))
```

This weight will appear in `model.parameters()`

8. Important Notes

- Always call `super().__init__()` in `__init__()`.
- Implement `forward()`; this is called when you use `model(input)`.
- All submodules and parameters are tracked automatically.

PyTorch nn.Linear and nn.Embedding

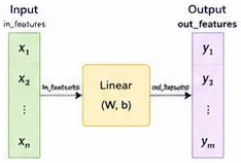
1. nn.Linear (Fully Connected Layer)

Definition

Applies a linear transformation to the incoming data:

$$y = xA^T + b$$

- **x** : input
- **A** : weight matrix of shape (out_features, in_features)
- **b** : bias vector of shape (out_features)



Syntax

```
torch.nn.Linear(in_features, out_features, bias=True, dtype=None, device=None)
```

- **in_features (int)**: size of each input sample
- **out_features (int)**: size of each output sample
- **bias (bool)**: if set to False, the layer will not learn an additive bias.

Input / Output Shape

Input Shape	Output Shape
(in_features)	(out_features)
(batch_size, in_features)	(batch_size, out_features)
(batch_size, *, in_features)	(batch_size, *, out_features)

* means any number of additional dimensions.

Example

```
import torch
import torch.nn as nn

# Create a Linear layer
linear = nn.Linear(in_features=4, out_features=3, bias=True)

# Input: batch of 2 samples, each with 4 features
x = torch.tensor([[1.0, 2.0, 3.0, 4.0],
                  [5.0, 6.0, 7.0, 8.0]])  # shape: (2, 4)

# Forward pass
output = linear(x)   # shape: (2, 3)
print(output)
```

Real-world Applications

- Used in almost every neural network.
- Classifier head in models (e.g., last layer before output).
- Mapping features to a new representation space.
- Used in MLPs, Transformers (in projection layers), etc.

Key Points

- ✅ Performs a linear transformation.
- ✅ Learns weights and (optional) bias.
- ✅ Does not apply activation (use separately).
- ✅ Core building block of dense layers.

Common Mistakes

- ❌ Forgetting that it expects the last dimension = in_features.
- ❌ Confusing batch size with in_features.
- ❌ Not applying activation when needed (Linear is only linear!).

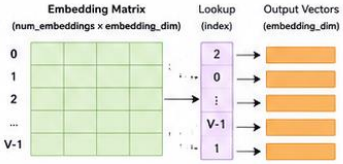
Think of nn.Linear as a calculator:
It takes numbers, does: output = input × weights + bias
and gives new numbers.

2. nn.Embedding (Embedding Layer)

Definition

Turns positive integer indices (usually word IDs) into dense vectors of fixed size.

It looks up embeddings from a learnable matrix.



Syntax

```
torch.nn.Embedding(num_embeddings, embedding_dim,
                   padding_idx=None, max_norm=None, dtype=None, device=None)
```

- **num_embeddings (int)**: size of the dictionary (vocabulary size).
- **embedding_dim (int)**: size of each embedding vector.
- **padding_idx** (int, optional): if specified, the embeddings at this index do not contribute to the gradient.

Input / Output Shape

Input Shape (indices)	Output Shape (embeddings)
(seq_len)	(seq_len, embedding_dim)
(batch_size, seq_len)	(batch_size, seq_len, embedding_dim)
(*)	(*, embedding_dim)

Example

```
import torch
import torch.nn as nn

vocab_size = 10    # total number of tokens
embedding_dim = 4

embedding = nn.Embedding(num_embeddings=vocab_size,
                         embedding_dim=embedding_dim)

# Input: batch of 2 sequences, each of length 3 (token indices)
x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])    # shape: (2, 3)

# Forward pass
output = embedding(x)   # shape: (2, 3, 4)
print(output)
```

Real-world Applications

- Word / token embeddings in NLP models.
- User ID / Item ID embeddings in recommendation systems.
- Any categorical data representation.
- Used in RNNs, Transformers, and many other architectures.

Key Points

- ✅ Maps token IDs to dense vectors.
- ✅ Learns embeddings during training.
- ✅ More efficient than one-hot encoding.
- ✅ Supports padding for variable length sequences.

Common Mistakes

- ❌ Passing one-hot vectors instead of indices.
- ❌ Forgetting padding_idx for padded sequences.
- ❌ Using wrong vocab size (out of range indices will crash).

Think of nn.Embedding as a dictionary:
You give an index (word ID), it returns the corresponding meaning vector (embedding).

Linear vs Embedding

Aspect	nn.Linear	nn.Embedding
Purpose	Applies linear transformation	Maps indices to dense vectors
Input	Continuous values (float)	Integer indices (long)
Weights Shape	(out_features, in_features)	(num_embeddings, embedding_dim)
Output	Transformed features	Embedding vectors
Use Cases	MLPs, Classifiers, Projections	NLP, Recommenders, Categorical data

When to Use What?

- Use nn.Linear when you have numerical features and want to transform them.
- Use nn.Embedding when you have discrete IDs (like words, user IDs, item IDs) and want dense representations.

PyTorch: The Role of forward() Method

1. DEFINITION

The forward() method defines how data flows through your model.

It contains the logic for computations from input(s) to output(s).

In short: forward() tells PyTorch what happens when you call the model.

2. WHY IS IT IMPORTANT?

- It defines the computation graph.
- It is called automatically when you call the model on input data.
- All layers, operations and activations are applied inside it.
- Autograd (automatic differentiation) tracks operations in forward() to compute gradients during backward().
- It makes your model reusable, clean and modular.

3. HOW IT WORKS (BEHIND THE SCENES)

Input is passed to model	When you do: `output = model(x)`
forward() is called	PyTorch internally calls: `model.forward(x)`
Operations in forward() are executed	All computations inside forward() build the computation graph.
Output is returned	The output is used to compute loss.
Loss is computed & backward() is called	During loss.backward(), PyTorch uses the graph from forward() to compute gradients for parameters.

4. 4. SYNTAX

```
class MyModel(nn.Module):
    def __init__(self):
        super()._init__()
        # define layers

    def forward(self, input1, input2=None):
        # operations
        output = ...
        return output
```

Takes input tensor(s) as arguments

Returns output tensor(s)

Always use forward(), not __call__() or other methods for defining model computation.

5. EXAMPLE

A simple MLP Model

```
import torch
import torch.nn as nn

class MLP(nn.Module):
   def __init__(self, in_features, hidden, out_features):
      super()._init__()
      self.fc1 = nn.Linear(in_features, hidden)
      self.relu = nn.ReLU()
      self.fc2 = nn.Linear(hidden, out_features)

  def forward(self, x):
      x = self.fc1(x)      # Linear layer
      x = self.relu(x)     # Activation
      x = self.fc2(x)      # Linear layer
      return x
```

Flow of Data

Input (batch_size, in_features)
↓
Linear (fc1)
↓
ReLU
↓
Linear (fc2)
↓
Output (batch_size, out_features)

Using the model

```
model = MLP(in_features=10, hidden=32, out_features=2)
x = torch.randn(8, 10)      # batch_size=8, in_features=10
out = model(x)              # forward() called automatically
print(out.shape)            # torch.Size([8, 2])
```

6. KEY POINTS ABOUT forward()

- **Defines the computation graph.** Every operation in forward() is tracked by Autograd.
- **Automatically called.** When you call model(x), forward(x) runs.
- **Can have any number of inputs/outputs.** return can be a tensor, tuple, list or dict.
- **Can use any PyTorch operations.** Layers, activations, math ops, control flow, loops, etc.
- **No need to call layers manually during training.** Just define them in __init__ and use in forward().

7 7. ADVANCED EXAMPLES

Multiple Inputs

```
class AddModel(nn.Module):
    def __init__(self):
        super()._init__()
        self.fc = nn.Linear(10, 10)

    def forward(self, x1, x2):
        x = x1 + x2
        return self.fc(x)
```

```
x1 = torch.randn(4, 10)
x2 = torch.randn(4, 10)
model = AddModel()
out = model(x1, x2)
```

Multiple Outputs

```
class MultiOut(nn.Module):
    def __init__(self):
        super()._init__()
        self.fc1 = nn.Linear(10, 5)
        self.fc2 = nn.Linear(10, 3)

    def forward(self, x):
        out1 = self.fc1(x)
        out2 = self.fc2(x)
        return out1, out2
```

```
x = torch.randn(2, 10)
model = MultiOut()
y1, y2 = model(x)
```

8. forward() vs __call__()

Aspect	forward()	__call__() (model())
Defined by user?	Yes	No (defined in nn.Module)
Purpose	Defines computation	Handles hooks, modes, then calls forward()
Called when you do model(x)?	Yes (internally)	Yes (this gets called)
Override?	Yes, you should	No, don't override

9. COMMON MISTAKES

- Forgetting to return output.
- Doing operations outside forward() that depend on input.
- Using .numpy() or .item() inside forward() (can break gradients).
- Modifying tensors in-place (e.g., x += 1) may cause issues.

10. BEST PRACTICES

- Keep forward() clean and readable.
- Use nn.Module layers in __init__ and apply in forward().
- Avoid in-place operations unless needed.
- Return tensors (or structures of tensors).
- Test forward() independently with dummy input.

11. REAL-WORLD USE CASES

Image Classification
Input image → CNN layers → class probabilities

NLP Models
Token IDs → Embedding → Transformer → Output

Object Detection
Image → Backbone → Head → Boxes, Classes

Generative Models
Noise/Prompt → Model → Generated Output

Time Series
Sequence → RNN/Transformer → Forecast/Prediction

QUICK SUMMARY

forward() is the heart of your PyTorch model.
It defines how inputs are transformed into outputs, builds the computation graph and enables learning through backpropagation.

ONE LINE TO REMEMBER

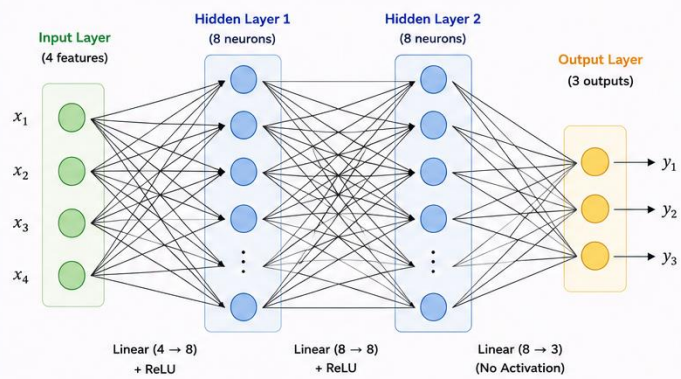
"You design the flow in forward(), PyTorch takes care of the rest."

PyTorch – Feedforward Neural Network (FNN)

PyTorch Code

```python
import torch
import torch.nn as nn
import torch.optim as optim

# --------------------------
# 1. Define the Feedforward Neural Network
# ----------------------------
class FeedforwardNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(FeedforwardNN, self).__init__()

        # 3 Layers: Input -> Hidden -> Output
        self.fc1 = nn.Linear(input_size, hidden_size)   # Input to Hidden
        self.relu = nn.ReLU()                           # Activation
        self.fc2 = nn.Linear(hidden_size, hidden_size)  # Hidden to Hidden
        self.fc3 = nn.Linear(hidden_size, output_size)  # Hidden to Output

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x

# --------------------------
# 2. Hyperparameters
# --------------------------
input_size  = 4    # Number of input features
hidden_size = 8    # Number of neurons in hidden layers
output_size = 3    # Number of output classes/values
learning_rate = 0.01

# --------------------------
# 3. Create Model, Loss, Optimizer
# --------------------------
model = FeedforwardNN(input_size, hidden_size, output_size)
criterion = nn.CrossEntropyLoss()  # For classification
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# --------------------------
# 4. Dummy Input and Forward Pass
x_dummy = torch.randn(5, input_size)   # Batch of 5 samples
output = model(x_dummy)
print("Output shape:", output.shape)   # (5, output_size)
```

Neural Network Architecture



Architecture Summary

- **Input Layer:** 4 neurons (features)
- **Hidden Layer 1:** 8 neurons + ReLU
- **Hidden Layer 2:** 8 neurons + ReLU
- **Output Layer:** 3 neurons (for 3 classes/values)
- Fully Connected between consecutive layers
- ReLU introduces non-linearity
- Output layer is linear (suitable for **CrossEntropyLoss**)

Legend

- Input Neurons
- Hidden Neurons
- Output Neurons
- Weighted Connections

Key Points

- ✓ Feedforward: Information flows in one direction (→)
- ☑ No cycles or loops
- ✓ Learns patterns from input to output using weights
- ☑ Backpropagation updates the weights

How It Works

Input (Batch of 5, 4 features) → Linear (4 → 8) → ReLU → Linear (8 → 8) → ReLU → Linear (8 → 3) → Output (Batch of 5, 3 outputs)

Output Shape

Since we passed a batch of 5 samples and `output_size = 3`, the output shape is: `(5, 3)`

LEARNING RATE SCHEDULING & GRADIENT CLIPPING IN PYTORCH

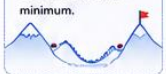
1. LEARNING RATE SCHEDULING

DEFINITION

Learning Rate Scheduling is a technique to change the learning rate during training instead of keeping it fixed. It helps the model converge faster and often leads to better performance.

INTUITION

Think of learning rate as the "step size" while walking down a valley (loss landscape). Big steps to move fast initially, smaller steps to not overshoot near the minimum.



WHY USE IT?

- Large LR → faster learning in the beginning
- Small LR → fine-tuning towards the end
- Helps escape plateaus and local minima
- Improves convergence and generalization

COMMON SCHEDULERS IN PYTORCH

Scheduler	What it does	Best used when
StepLR	Decays LR by a factor every few epochs	Simple problems
MultiStepLR	Decays LR at specific milestones	When you know important epochs
ExponentialLR	Decays LR exponentially every epoch	Smooth decay needed
CosineAnnealingLR	Cosine decay from max LR to min LR	Good for most tasks
ReduceLROnPlateau	Reduces LR when metric has stopped improving	Validation loss plateaus

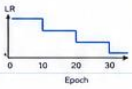
SYNTAX (GENERAL WORKFLOW)

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
          # create optimizer
scheduler = torch.optim.lr_scheduler.<SchedulerName>(optimizer, ...)
          # create scheduler

for epoch in range(num_epochs):
    train()
    validate()

    # Step the scheduler
    scheduler.step()  # for most schedulers
    # or
    # scheduler.step(validation_loss)  # for ReduceLROnPlateau
```

EXAMPLES

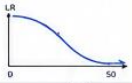
1. StepLR – Decays LR every step_zize epochs by gamma

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer,
                                            step_size=10, gamma=0.1)
```



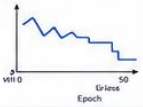
2. CosineAnnealingLR – Cosine decay to eta_min

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=50, eta_min=1e-6)
```



3. ReduceLROnPlateau – Reduce LR when val loss stops improving

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=5,
    min_lr=1e-6, verbose=True)
```



TIPS & BEST PRACTICES

- ✔ Warm-up the learning rate for first few epochs (especially for Transformers).
- ✔ Don't decay too early – let the model learn meaningful patterns first.
- ✔ Use **ReduceLROnPlateau** when you are not sure about the schedule.
- ✔ Monitor LR during training to understand its effect.

COMMON MISTAKES

- ✖ Decaying LR too aggressively.
- ✖ Forgetting to call scheduler-step().
- ✖ Using a scheduler not suitable for your problem.

REAL-WORLD USE CASES

- CNNs for Image Classification – StepLR / CosineAnnealingLR are widely used.
- NLP / Transformers – Warmup + Linear Decay or Cosine Decay is standard.
- Fine-tuning Pretrained Models – Lower initial LR + scheduler for stability.
- Training GANs – Often use different schedulers for G and D.

2. GRADIENT CLIPPING

DEFINITION

Gradient Clipping is a technique to prevent exploding gradients by restricting the norm or value of gradients during backpropagation.

INTUITION

If gradients are too large, the model updates become too big and overshoot the optimal point. Clipping acts like a "speed limit" for gradient updates.

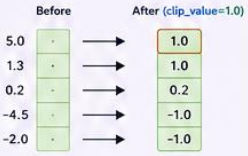
WHY USE IT?

- Prevents unstable updates due to huge gradients
- Especially useful in RNNs, LSTMs, Transformers
- Leads to more stable and faster training

TYPES OF GRADIENT CLIPPING

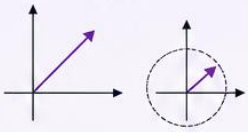
1. Value Clipping (Clip by value)

Clips each gradient element to be within [−clip_value, +clip_value]



2. Norm Clipping (Clip by norm)

Clips the gradient vector so that its norm (L2 norm) does not exceed max_norm.



Before (norm > max_norm) — After (norm = max_norm)

SYNTAX

```
# 1. Value Clipping
torch.nn.utils.clip_grad_value_(
   parameters, clip_value)
```

```
# 2. Norm Clipping
torch.nn.utils.clip_grad_norm_(
   parameters, max_norm,
   norm_type=2.0)
```

WORKFLOW (TYPICAL TRAINING LOOP)

```
for data, target in loader:
    optimizer.zero_grad()
    output = model(data)
    loss = criterion(output, target)
    loss.backward()

    # ---- Gradient Clipping ----
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    # or
    # torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=1.0)
    optimizer.step()
```

EXAMPLE (NORM CLIPPING)

```
import torch
import torch.nn as nn

model = nn.LSTM(input_size=128, hidden_size=256, num_layers=2)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
max_norm = 1.0

for data, target in loader:
    optimizer.zero_grad()
    output, _ = model(data)
    loss = criterion(output, target)
    loss.backward()

    # Clip gradients by global norm
    total_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm=max_norm)
    optimizer.step()
```

clip_grad_norm_ returns the total norm of gradients (before clipping). Useful for monitoring.

TIPS & BEST PRACTICES

- ✔ Use **Norm Clipping** for RNNs / Transformers.
- ✔ Common **max_norm** values: **0.1, 0.5, 1.0, 5.0.**
- ✔ Apply clipping after loss.backward() and **before optimizer.step().**
- ✔ Monitor gradient norms during training.

COMMON MISTAKES

- ✖ Clipping before backward().
- ✖ Using too small max_norm → very slow learning.
- ✖ Not needed in most CNNs (usually safe to ignore).

REAL-WORLD USE CASES

- **RNN / LSTM / GRU Models** – Prevent exploding gradients.
- **Transformer Models** – Widely used (max_norm=1.0 is common).
- Seq2Seq, Language Models, Speech Models – Stabilize training.
- **Reinforcement Learning** – Helps in policy/value network stability.

QUICK REFERENCE

```
for param_group in optimizer.param_groups:
    print(param_group['lr'])
```

Order of Calls (per epoch)

```
train() / forward / backward
          ↓
   optimizer.step()
          ↓
   scheduler.step()
```

Common Schedulers

StepLR, MultiStepLR, ExponentialLR, CosineAnnealingLR, CosineAnnealingWarmRestarts, ReduceLROnPlateau, LambdaLR, OneCycleLR

KEY TAKEAWAY

Use **Learning Rate Scheduling** to control how much the model learns over time.
Use **Gradient Clipping** to control how much the model updates at each step.

BEST COMBINATION

Learning Rate Scheduling + Gradient Clipping
= Stable, Fast and Better Converging Models.

Write a pytorch fuinction that manually calculates the gradient for a linear regression model

```
import torch

# ==========================================
# 1. Sample Dataset
# ==========================================

# Input features
x = torch.tensor([1.0, 2.0, 3.0, 4.0])

# Ground truth labels
y = torch.tensor([2.0, 4.0, 6.0, 8.0])

# ==========================================
# 2. Initialize Parameters
# ==========================================

# Random initial weight and bias
w = torch.tensor(0.0)
b = torch.tensor(0.0)

# Learning rate
lr = 0.01

# Number of samples
n = len(x)

# ==========================================
# 3. Training Loop
# ==========================================

for epoch in range(100):

    # -------------------------------------
    # Forward Pass
    # -------------------------------------

    # Linear Regression Equation
    y_pred = w * x + b

    # Mean Squared Error Loss
    loss = ((y_pred - y) ** 2).mean()
```

```
    # -------------------------------------
    # Manual Gradient Calculation
    # -------------------------------------

    # dL/dw
    dw = (2/n) * torch.sum((y_pred - y) * x)

    # dL/db
    db = (2/n) * torch.sum(y_pred - y)

    # -------------------------------------
    # Parameter Update
    # -------------------------------------

    w = w - lr * dw
    b = b - lr * db

    # -------------------------------------
    # Print Progress
    # -------------------------------------

    if epoch % 10 == 0:
        print(f"Epoch {epoch}")
        print(f"Loss  : {loss:.4f}")
        print(f"Weight: {w:.4f}")
        print(f"Bias  : {b:.4f}")
        print("-" * 30)

# ==========================================
# 4. Final Prediction
# ==========================================

print("\nFinal Parameters")
print(f"Weight = {w:.4f}")
print(f"Bias   = {b:.4f}")
```

Implement linear regression using pytorch

```
import torch
import torch.nn as nn
import torch.optim as optim

x = torch.tensor([[1.0],
                  [2.0],
                  [3.0],
                  [4.0]])

y = torch.tensor([[2.0],
```

```
                   [4.0],
                   [6.0],
                   [8.0]])

class LinearRegressionModel(nn.Module):

    def __init__(self):
        super(LinearRegressionModel, self).__init__()

        self.linear = nn.Linear(
            in_features=1,
            out_features=1
        )

    def forward(self, x):

        return self.linear(x)


model = LinearRegressionModel()

criterion = nn.MSELoss()

optimizer = optim.SGD(
    model.parameters(),
    lr=0.01
)

epochs = 1000

for epoch in range(epochs):

    predictions = model(x)

    loss = criterion(predictions, y)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    if epoch % 100 == 0:

        print(f"Epoch [{epoch}/{epochs}]")
        print(f"Loss: {loss.item():.4f}")

with torch.no_grad():

    predicted = model(torch.tensor([[5.0]]))
```

```
    print("\nPrediction for x = 5:")

    print(predicted.item())
```

Create a simple feed forward neural network in pytorch that works on MNIST dataset

```
# ================================
# Feed Forward Neural Network on MNIST using PyTorch
# ================================

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# ------------------------------------------------
# 1. Device Configuration
# ------------------------------------------------
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# ------------------------------------------------
# 2. Hyperparameters
# ------------------------------------------------
input_size = 28 * 28     # MNIST images are 28x28
hidden_size = 128
num_classes = 10
batch_size = 64
learning_rate = 0.001
epochs = 5

# ------------------------------------------------
# 3. Load MNIST Dataset
# ------------------------------------------------

# Transform:
# Converts image to tensor
transform = transforms.ToTensor()

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)
```

```python
test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    transform=transform,
    download=True
)

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=batch_size,
    shuffle=True
)

test_loader = DataLoader(
    dataset=test_dataset,
    batch_size=batch_size,
    shuffle=False
)

# ------------------------------------------------
# 4. Create Feed Forward Neural Network
# ------------------------------------------------

class FeedForwardNN(nn.Module):

    def __init__(self):
        super(FeedForwardNN, self).__init__()

        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()

        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):

        # x shape:
        # (batch_size, 1, 28, 28)

        # Flatten image into vector
        x = x.reshape(x.shape[0], -1)

        # First Linear Layer
        x = self.fc1(x)

        # Activation Function
        x = self.relu(x)

        # Output Layer
        x = self.fc2(x)
```

```
        return x


model = FeedForwardNN().to(device)

print(model)

# ------------------------------------------------
# 5. Loss Function and Optimizer
# ------------------------------------------------

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=learning_rate
)

# ------------------------------------------------
# 6. Training Loop
# ------------------------------------------------

for epoch in range(epochs):

    model.train()

    running_loss = 0.0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        # ------------------------------
        # Forward Pass
        # ------------------------------
        outputs = model(images)

        loss = criterion(outputs, labels)

        # ------------------------------
        # Backward Pass
        # ------------------------------
        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        running_loss += loss.item()
```

```
    avg_loss = running_loss / len(train_loader)

    print(f"Epoch [{epoch+1}/{epochs}] Loss: {avg_loss:.4f}")

# ------------------------------------------------
# 7. Evaluation
# ------------------------------------------------

model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        # Get class with highest probability
        _, predictions = torch.max(outputs, dim=1)

        total += labels.size(0)

        correct += (predictions == labels).sum().item()

accuracy = 100 * correct / total

print(f"\nTest Accuracy: {accuracy:.2f}%")
```

Write code for image classificatio using CNN in Pytorch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Using Device:", device)

batch_size = 64
learning_rate = 0.001
```

```
epochs = 5

transform = transforms.ToTensor()

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    transform=transform,
    download=True
)

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=batch_size,
    shuffle=True
)

test_loader = DataLoader(
    dataset=test_dataset,
    batch_size=batch_size,
    shuffle=False
)

class CNN(nn.Module):

    def __init__(self):
        super(CNN, self).__init__()

        self.conv1 = nn.Conv2d(
            in_channels=1,
            out_channels=16,
            kernel_size=3,
            stride=1,
            padding=1
        )

        self.pool = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        self.conv2 = nn.Conv2d(
            in_channels=16,
```

```
            out_channels=32,
            kernel_size=3,
            stride=1,
            padding=1
        )

        self.fc1 = nn.Linear(32 * 7 * 7, 128)

        self.relu = nn.ReLU()

        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):

        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)

        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x)

        x = x.reshape(x.shape[0], -1)

        x = self.fc1(x)
        x = self.relu(x)

        x = self.fc2(x)

        return x


model = CNN().to(device)

print(model)

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=learning_rate
)

for epoch in range(epochs):

    model.train()

    running_loss = 0.0

    for images, labels in train_loader:
```

```
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        loss = criterion(outputs, labels)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)

    print(f"Epoch [{epoch+1}/{epochs}]")
    print(f"Loss: {avg_loss:.4f}")

model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        _, predicted = torch.max(outputs, dim=1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

print(f"\nTest Accuracy: {accuracy:.2f}%")
```